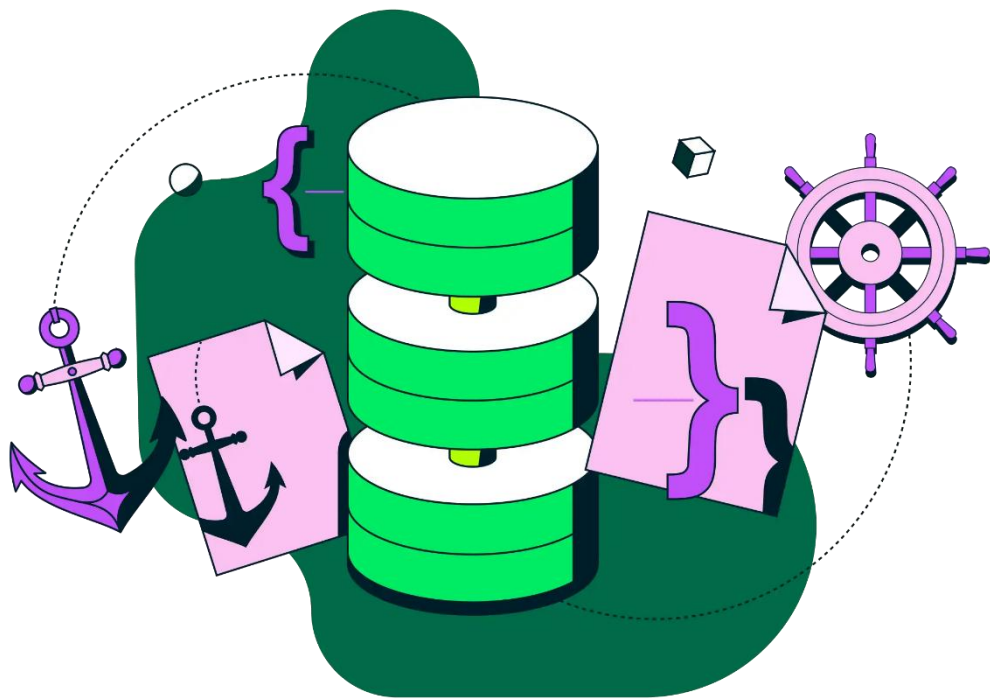


1 DE DICIEMBRE DE 2025



AE U2.6 - DESPLIEGUE EN MONGODB + U2.7 ROOTER

REALIZADO POR DANIEL GARCÍA MÉNDEZ

Contenido

Introducción al modelado del proyecto	2
Modelado de datos en E/R.....	2
Modelo adaptado a MongoDB.....	10
Cambios al pasar de E/R a MongoDB.....	17

Introducción al modelado del proyecto

En esta tarea veremos la estructura de datos de mi proyecto llamada Gimnasio Inteligente, en este documento veremos todas las entidades necesarias para el proyecto, así como las relaciones entre ellas.

Por otra parte, veremos el paso del modelo relacional al modelo no relacional en MongoDB, trabajaremos con Mongoose en nodeJs para crear e insertar los datos en las colecciones.

Modelado de datos en E/R

La entidad central es **Usuario**, que se especializa en tres tipos: **Cliente**, **Instructor** y **Administrador**. A partir de esta base se conectan las demás entidades que reflejan las operaciones clave del gimnasio: clases, calendario, productos, ventas, asistencia, informes mensuales y cargas masivas.

Estas son las clases principales de mi modelo E/R junto a sus relaciones y cardinalidades que adaptaré en el siguiente apartado:

Entidades principales

Usuario

Atributos:

- **DNI (PK)**
- Nombre_Completo
- Email
- Contraseña
- Fecha_De_Registro

Descripción: Representa a cada persona registrada en el sistema. Incluye datos mínimos necesarios para identificar al usuario y proteger su cuenta.

Cliente

Atributos:

- **ID_Cliente (PK)**
- DNI (FK Usuario)
- Objetivo_Físico
- Fecha_Alta
- Membresía (activa / vencida)

Descripción: Representa a los usuarios que son clientes del gimnasio, con información sobre sus metas y estado de membresía.

Instructor

Atributos:

- **ID_Instructor (PK)**
- DNI (FK Usuario)
- Especialidad
- Perfil_Profesional

Descripción: Representa a los usuarios que son instructores, con datos sobre su especialidad y perfil profesional.

Administrador

Atributos:

- **ID_Admin (PK)**
- DNI (FK Usuario)

Descripción: Representa a los usuarios con rol de administrador, responsables de la gestión global del sistema.

Clase

Atributos:

- **ID_Clase (PK)**
- Nombre
- Tipo
- Horario
- Nivel
- Cupo_Max
- ID_Instructor (FK)

Descripción: Representa las actividades impartidas en el gimnasio, con su respectivo instructor y cupo máximo.

Calendario

Atributos:

- **ID_Calendario (PK)**
- ID_Clase (FK)
- Fecha
- Estado (aprobada / pendiente)

Descripción: Permite gestionar las clases programadas y su estado dentro del sistema.

Producto

Atributos:

- **ID_Producto (PK)**
- Nombre
- Precio
- Stock

Descripción: Representa los productos disponibles para la venta en el gimnasio.

Venta

Atributos:

- **ID_Venta (PK)**
- ID_Cliente (FK)
- ID_Producto (FK)
- Fecha
- Cantidad
- Total

Descripción: Registra las compras realizadas por los clientes, incluyendo cantidad y total de la operación.

Asistencia

Atributos:

- **ID_Asistencia (PK)**
- ID_Cliente (FK)
- ID_Clase (FK)
- Fecha
- Estado (presente / ausente)

Descripción: Controla la participación de los clientes en las clases.

InformeMensual

Atributos:

- **ID_Informe (PK)**
- ID_Cliente (FK)
- Fecha_Generación
- Clases_Asistidas
- Evolución_RM
- Productos_Comprados

Descripción: Genera métricas de actividad y evolución de cada cliente en un informe mensual.

CargaMasiva

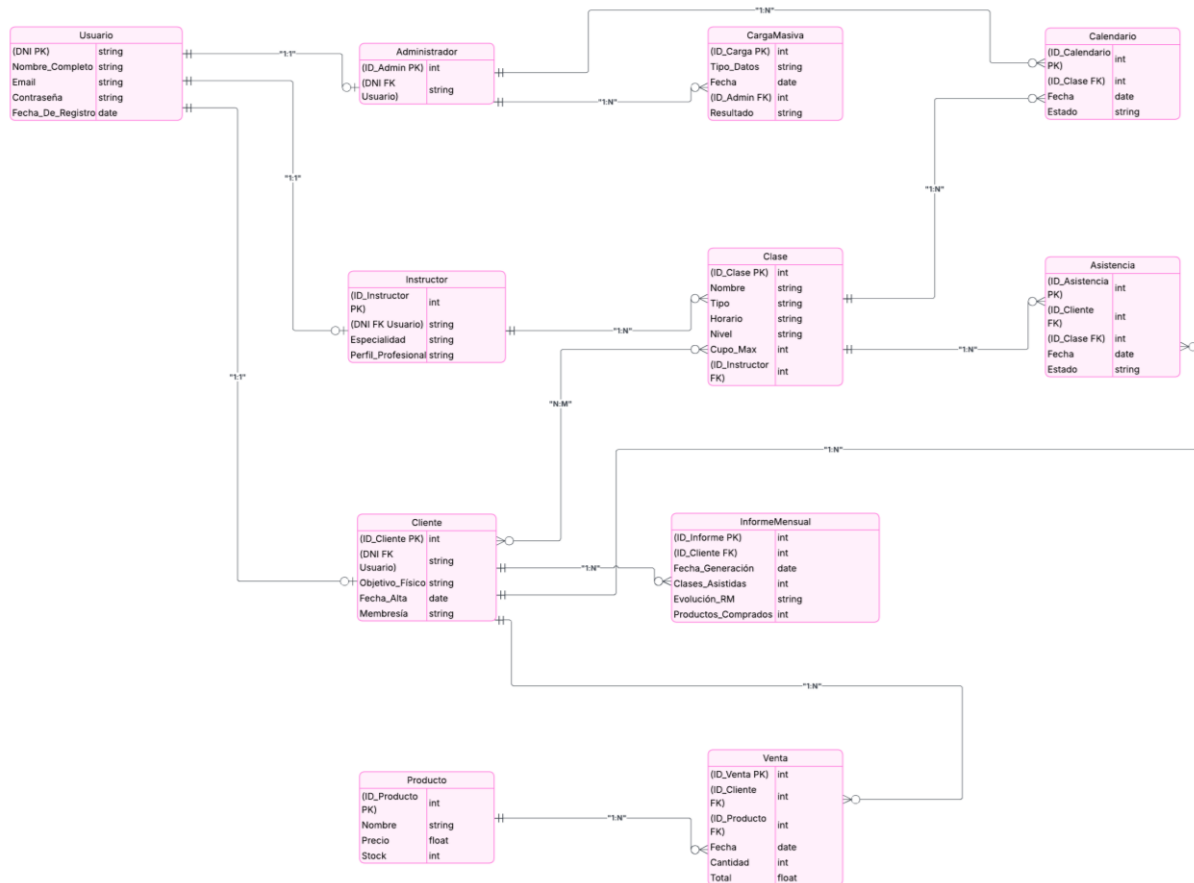
Atributos:

- **ID_Carga (PK)**
- Tipo_Datos (usuarios, productos, clases)
- Fecha
- ID_Admin (FK)
- Resultado (correctos / rechazados)

Descripción: Registra las operaciones de importación masiva de datos realizadas por los administradores.

Modelo Entidad-Relación de la aplicación Gimnasio inteligente

En esta imagen podrás observar un breve resumen en forma de diagrama de E/R todas las entidades con sus atributos y relaciones.



Relaciones

Usuario — Cliente / Instructor / Administrador

Tipo de relación: 1:1 (especialización)

Explicación: Un usuario identificado por su DNI, puede ser cliente, instructor o administrador.

Cardinalidad:

- Usuario (1,1) ↔ Cliente (0,1)
- Usuario (1,1) ↔ Instructor (0,1)
- Usuario (1,1) ↔ Administrador (0,1)

Cliente — Clase

Tipo de relación: N:M

Explicación: Un cliente puede apuntarse a varias clases y una clase puede tener varios clientes.

Cardinalidad:

- Cliente (0,N) ↔ Clase (0,N)
- Cliente (1,N) ↔ Asistencia (1,1)
- Clase (1,N) ↔ Asistencia (1,1)

Instructor — Clase

Tipo de relación: 1: N

Explicación: Un instructor puede impartir varias clases, pero cada clase tiene un único instructor asignado.

Cardinalidad:

- Instructor (1,1) ↔ Clase (0,N)

Clase — Calendario

Tipo de relación: 1: N

Explicación: Cada clase puede tener varias programaciones en el calendario (ej. diferentes fechas). Cada entrada de calendario pertenece a una única clase.

Cardinalidad:

- Clase (1,1) ↔ Calendario (0,N)

Administrador — Calendario**Tipo de relación:** 1: N**Explicación:** Un administrador puede aprobar varias clases en el calendario.**Cardinalidad:**

- Administrador (1,1) ↔ Calendario (0, N)

Cliente — Venta**Tipo de relación:** 1: N**Explicación:** Un cliente puede realizar varias compras, pero cada venta pertenece a un único cliente.**Cardinalidad:**

- Cliente (1,1) ↔ Venta (0,N)

Producto — Venta**Tipo de relación:** 1: N**Explicación:** Un producto puede estar en varias ventas, pero cada venta corresponde a un único producto.**Cardinalidad:**

- Producto (1,1) ↔ Venta (0,N)

Cliente — Asistencia**Tipo de relación:** 1: N**Explicación:** Un cliente puede tener múltiples registros de asistencia, pero cada registro pertenece a un único cliente.**Cardinalidad:**

- Cliente (1,1) ↔ Asistencia (0,N)

Clase — Asistencia**Tipo de relación:** 1: N**Explicación:** Una clase puede tener varios registros de asistencia, pero cada registro corresponde a una sola clase.**Cardinalidad:**

- Clase (1,1) ↔ Asistencia (0,N)

Cliente — InformeMensual**Tipo de relación:** 1: N**Explicación:** Cada cliente puede tener varios informes mensuales, pero cada informe pertenece a un único cliente.**Cardinalidad:**

- Cliente (1,1) ↔ InformeMensual (0,N)

Administrador — CargaMasiva**Tipo de relación:** 1: N**Explicación:** Un administrador puede realizar varias cargas masivas, pero cada carga pertenece a un único administrador.**Cardinalidad:**

- Administrador (1,1) ↔ CargaMasiva (0,N)

Tabla del modelo lógico

Tabla	Atributos	PK	FK
Usuario	DNI, Nombre_Completo, Email, Contraseña, Fecha_De_Registro	DNI	-
Cliente	ID_Cliente, DNI, Objetivo_Físico, Fecha_Alta, Membresía	ID_Cliente	DNI → Usuario (DNI)
Instructor	ID_Instructor, DNI, Especialidad, Perfil Profesional	ID_Instructor	DNI → Usuario (DNI)
Administrador	ID_Admin, DNI	ID_Admin	DNI → Usuario(DNI)
Clase	ID_Clase, Nombre, Tipo, Horario, Nivel, Cupo_Max, ID_Instructor	ID_Clase	ID_Instructor → Instructor(ID_Instructor)
Calendario	ID_Calendario, ID_Clase, Fecha, Estado	ID_Calendario	ID_Clase → Clase(ID_Clase)
Producto	ID_Producto, Nombre, Precio, Stock	ID_Producto	-
Venta	ID_Venta, ID_Cliente, ID_Producto, Fecha, Cantidad, Total	ID_Venta	ID_Cliente → Cliente(ID_Cliente), ID_Producto → Producto(ID_Producto)

Asistencia	ID_Asistencia, ID_Cliente, ID_Clase, Fecha, Estado	ID_Asistencia	ID_Cliente Cliente(ID_Cliente), ID_Clase Clase(ID_Clase)	→
InformeMensual	ID_Informe, ID_Cliente, Fecha_Generación, Clases_Asistidas, Evolución_RM, Productos_Comprados	ID_Informe	ID_Cliente Cliente(ID_Cliente)	→
CargaMasiva	ID_Carga, Tipo_Datos, ID_Admin, Fecha, Resultado	ID_Carga	ID_Admin Administrador (ID_Admin)	→

Modelo adaptado a MongoDB

Para cada entidad, modelaré un JSON con cada uno de sus atributos principales que actuará de tabla.

Ejemplos:

Usuario: Su clave principal será el `_id`, que será el dni en este caso ya que será un atributo único.

```
{
  "_id": "DNI",
  "Nombre_Completo": "string",
  "Email": "string",
  "Contraseña": "string",
  "Fecha_De_Registro": "date"
}
```

```
{
```

```
_id: DNI
Nombre_Completo: string
Email: string
Contraseña: string
Fecha_De_Registro: date
```

Clientes: Aquí pondremos el `_id` como el `id_Cliente` que actuará como clave primaria, ya que será nuestra clave principal, luego pondremos como llave foránea el `dni`, que tendrá relación con el `dni` de usuario.

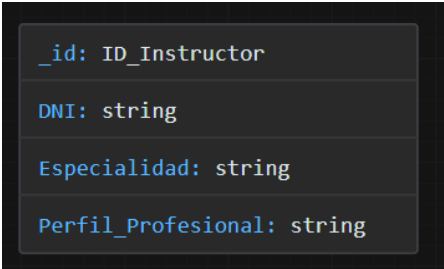
También pondremos de forma adicional su informe mensual, este tendrá como atributos la fecha, número de clases asistidas, un campo de evolución `rm` que será un texto resumiendo la evolución del récord del cliente en ejercicios, número de productos comprados y la fecha de generación del informe.

```
{
  "_id": "ID_Cliente",
  "DNI": "string",
  "Objetivo_Físico": "string",
  "Fecha_Alta": "date",
  "Membresía": "activa|vencida",
  "informesMensuales": [
    {
      "Clases_Asistidas": int,
      "Evolución_RM": "string",
      "Productos_Comprados": int,
      "Fecha_Generación": "date"
    }
  ]
}
```



Instructores: Aquí pondremos el `_id` que será el `id_Instructor` ya que será nuestra clave principal, luego pondremos como llave foránea el `dni`, que tendrá relación con el `dni` de usuario.

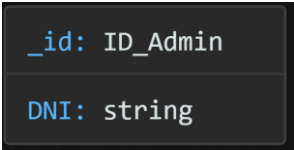
```
{
  "_id": "ID_Instructor", // Clave primaria
  "DNI": "string",        // Clave foránea → usuarios._id
  "Especialidad": "string",
  "Perfil_Profesional": "string"
}
```



```
_id: ID_Instructor
DNI: string
Especialidad: string
Perfil_Profesional: string
```

Administrador: El `_id` será el `Id_admin` que será un campo único en esta tabla, luego tendremos el `dni`, este hará de relación con el campo `DNI` de usuario.

```
{
  "_id": "ID_Admin",
  "DNI": "string"
}
```

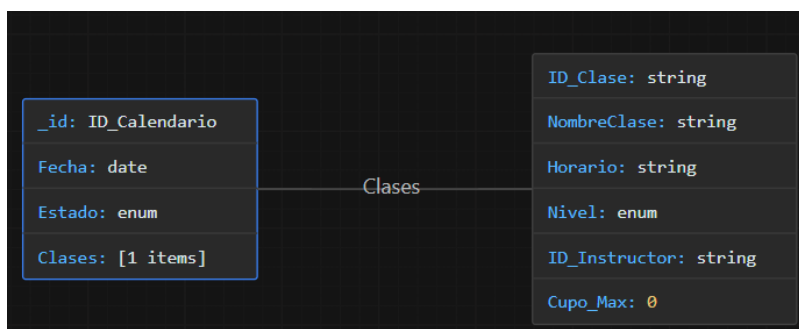


```
_id: ID_Admin
DNI: string
```

Calendario: Esta tabla tendrá un `_id` que será el campo que identificará al calendario, un campo tipo fecha, el estado del calendario que será un tipo enumerado (activo, pendiente, pasado) y una lista de clases.

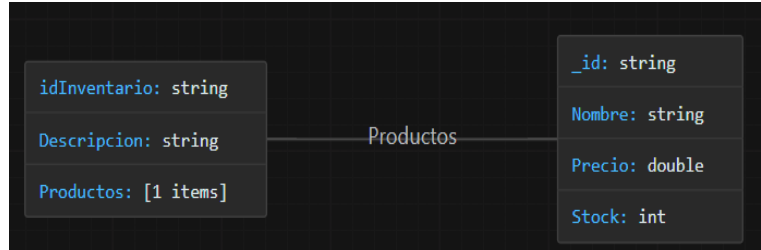
Cada **clase** tendrá los atributos `id_clase` que identificará a la clase, nombre, horario que será de tipo string, por ejemplo “M X J 8:00, V S 10:00”, nivel que será enum (principiante, intermedio, avanzado), el id del instructor que la da y el cupo máximo de la clase que será de tipo entero.

```
{
  "_id": "ID_Calendario",
  "Fecha": "date",
  "Estado": "enum",
  "Clases": [
    {
      "ID_Clase": "string",
      "NombreClase": "string",
      "Horario": "string",
      "Nivel": "enum",
      "ID_Instructor": "string",
      "Cupo_Max": "int"
    }
  ]
}
```



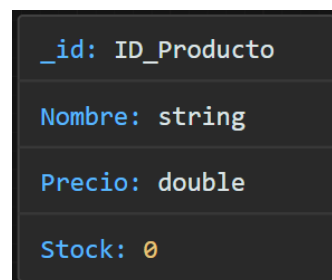
Inventario: Aquí pondremos `_id` que será el id del inventario y una lista de productos.

```
{
  "idInventario": "string",
  "Descripcion": "string",
  "Productos": [
    {
      "_id": "string",
      "Nombre": "string",
      "Precio": "double",
      "Stock": "int"
    }
  ]
}
```



Producto: El `_id` será el `id_Producto` que será un campo único en esta tabla, tendrá como atributos el nombre del producto, su precio y el stock. Con el stock ya no necesitamos una tabla adicional de inventario para saber si están disponibles.

```
{
  "_id": "ID_Producto",
  "Nombre": "string",
  "Precio": "double",
  "Stock": "int"
}
```



Ventas: Esta tabla tendrá un `_id` que será el id de la venta, junto a la fecha y una lista de Productos cuyos atributos serán el `id_producto`, nombre, el precio unitario, cantidad, el `PrecioTotalProducto` que será la multiplicación de la cantidad por el precio unitario y por último tendremos el atributo `TotalSumaListaProductos` que sumará todos los productos de la lista.

```
{
  "_id": "ID_Venta",          // Clave primaria de la venta
  "ID_Cliente": "string",     // ID del cliente que compra
  "Fecha": "date",
  "Productos": [
    {
      "ID_Producto": "string",
      "Nombre": "string",
      "PrecioUnitario": double,
      "Cantidad": int,
      "PrecioTotalProducto": double
    }
  ],
  "SumaTotalListaProductos": double
}
```



Asistencia: El `_id` será el `Id_asistencia` que hará de clave primaria, luego tendremos dos campos que harán distintas relaciones, el `id_cliente` hará una relación con la tabla Cliente y el `id_clase` hará una relación con la tabla Clase. También contará con un estado de tipo enum (presente, ausente).

```
{
  "_id": "ID_Asistencia",
  "ID_Cliente": "string",
  "ID_Clase": "string",
  "Fecha": "date",
  "Estado": "enum"
}
```

<code>_id:</code>	<code>ID_Asistencia</code>
<code>ID_Cliente:</code>	<code>string</code>
<code>ID_Clase:</code>	<code>string</code>
<code>Fecha:</code>	<code>date</code>
<code>Estado:</code>	<code>enum</code>

CargaMasiva: El `_id` será el `idCarga` que hará de clave primaria y tendremos una relación gracias al atributo `Id_admin` de la tabla de Admin. Luego tendremos el tipo de dato que será un enumerado (Usuarios, Clientes, Admins, Instructores, Ventas, Productos,), la fecha, el id del administrador y un resultado de tipo booleano como resultado que confirmará la inserción de toda la carga.

```
{
  "_id": "ID_Carga",
  "Tipo_Datos": "enum",
  "Fecha": "date",
  "ID_Admin": "string",
  "Resultado": "bool"
}
```

<code>_id:</code>	<code>ID_Carga</code>
<code>Tipo_Datos:</code>	<code>enum</code>
<code>Fecha:</code>	<code>date</code>
<code>ID_Admin:</code>	<code>string</code>
<code>Resultado:</code>	<code>bool</code>

Cambios al pasar de E/R a MongoDB

Usuario

En el modelo E/R, Usuario era la entidad central. En MongoDB se mantiene como colección independiente, usando el **DNI como _id**, para identificar al usuario.

Cliente

Antes tenía relación 1:1 con Usuario. En MongoDB se convierte en colección propia con **_id = ID_Cliente** y referencia al DNI de Usuario. El **InformeMensual**, que era entidad aparte, ahora está dentro de Cliente como un array.

Instructor

Se mantiene como colección independiente con **_id = ID_Instructor** y referencia al DNI de Usuario.

Administrador

En MongoDB se define como colección con **_id = ID_Admin** y referencia al DNI de Usuario.

Calendario

En el modelo E/R estaba ligado a Clase. En MongoDB se mete directamente una lista de clases dentro de cada documento de Calendario, junto con un campo Estado (activo, pendiente, pasado).

Inventario

Se crea como colección con **idInventario** y una lista de productos dentro. Aunque existe Inventario, también se mantiene la colección **Producto** independiente con stock, hice esta tabla adicional como catálogo donde los administradores, instructores pueden ver los productos disponibles.

Producto

Se mantiene como colección propia con **_id = ID_Producto**, nombre, precio y stock.

Venta

En el modelo E/R relacionaba Cliente y Producto. En MongoDB se meten los productos dentro de cada venta, con sus cantidades y totales, eliminando la tabla intermedia.

Asistencia

Se conserva como colección independiente, con referencias a Cliente y Clase. Se añade un campo Estado (presente/ausente).

InformeMensual

Ya no es colección independiente. Ahora está en Cliente como array de informes.

CargaMasiva

Se mantiene como colección relacionada con Administrador. El campo Resultado se simplifica a tipo booleano.

[Enlace de Github](#) con lo último que hicimos de Pokémon.